

Tecnológico de Monterrey

Clasificación de uso de suelo en imágenes satelitales EuroSAT con redes neuronales

Diseño de redes neuronales y aprendizaje profundo Gpo. 201

Lenin Adolfo Caballero Colunga - A01286929
Rafael Emilio Funez Perdomo - A00841342
Miguel Angel Hernández Ochoa - A00830949
Aldo Javier Rodríguez Maytorena - A01741554
Landon Mauricio Treviño Ortiz - A01199312

Eq. 3

Dra. Lorena B. Martínez Elizalde
Tecnológico de Monterrey - Campus Monterrey

24/10/2026

Resumen ejecutivo

El monitoreo del uso de suelo es clave para la planeación urbana, la agricultura y el seguimiento ambiental, pero clasificar imágenes satelitales a mano es lento y costoso. En este proyecto desarrollamos y comparamos redes neuronales para clasificar automáticamente imágenes satelitales según su uso y cobertura del suelo. Trabajamos con EuroSAT RGB, un conjunto de 27,000 imágenes de 64 × 64 píxeles distribuidas en diez clases, y comenzamos con una red densa como modelo base y posteriormente evaluamos una red convolucional y consideramos la transferencia de aprendizaje como alternativa, sin implementarla. Comparamos su desempeño mediante exactitud, precisión, recall, F1 y matriz de confusión, y evaluamos el aumento de datos como técnica para reducir el sobreajuste. En el conjunto de prueba (4,050 imágenes), la CNN con aumento de datos alcanzó 93.01 % de exactitud y un F1 macro de 0.928, frente a 45.19 % y 0.418 de la red densa, con menos de la mitad de los parámetros (1,337,898 contra 3,180,170). La transferencia de aprendizaje con ResNet-50 se consideró, pero no se implementó. Los resultados corresponden a una sola semilla y una sola partición de los datos.

Contexto del problema

El monitoreo del uso de suelo es una herramienta esencial para la gestión sostenible del territorio, la planificación urbana y el control ambiental. Sin embargo, la clasificación manual de imágenes satelitales representa un desafío por su alto costo y el tiempo que requiere. En este proyecto se buscó automatizar ese proceso mediante el desarrollo de una red neuronal profunda capaz de identificar distintos tipos de uso y cobertura del suelo a partir de imágenes satelitales.

Los satélites Sentinel-2 del programa Copernicus capturan imágenes con resolución de 10 m en las bandas visibles y pasan por la misma zona cada cinco días aproximadamente (Agencia Espacial Europea [ESA], s. f.). Cada imagen de EuroSAT, de 64 × 64 píxeles, cubre así unos 640 × 640 m, y la superficie de un país o una región genera miles de parches que no es práctico etiquetar a mano.

Para ello, se empleó el conjunto de datos EuroSAT, compuesto por aproximadamente 27,000 imágenes RGB de 64x64 pixeles capturadas por el satélite Sentinel-2, organizadas en diez clases que representan diferentes categorías de terreno (Helber et al., 2019). El trabajo comenzó con un modelo denso básico y avanzó hacia una arquitectura convolucional, aprovechando su capacidad para reconocer patrones espaciales y texturas en las imágenes.

En conjunto, este proyecto busca sentar las bases de un sistema de clasificación automática que apoye decisiones urbanas, agrícolas y ambientales.

Descripción del conjunto de datos

Para este proyecto se eligió EuroSAT, un conjunto de datos presentado por Helber et al. (2019) para clasificar el uso y la cobertura del suelo. Contiene 27,000 imágenes satelitales de distintas regiones de Europa, obtenidas mediante Sentinel-2, del programa Copernicus. Cada imagen tiene una etiqueta que indique el tipo de terreno al que pertenece.

Trabajamos con la versión RGB, formada por imágenes a color de 64 × 64 px y tres canales: rojo, verde y azul. Este tamaño resulta adecuado para el proyecto porque permite trabajar con imágenes pequeñas sin tener que reducirlas antes de usarlas. EuroSAT RGB está disponible en Kaggle y en TensorFlow Datasets; en este proyecto se descargó desde Kaggle y se trabajó con PyTorch. Esta versión contiene las 27,000 imágenes en un único conjunto, por lo que definimos una separación para entrenamiento, validación y prueba antes de comparar los modelos (TensorFlow, 2026).

El conjunto incluye 10 clases: cultivos anuales, bosques, vegetación herbácea, carreteras, zonas industriales, pastizales, cultivos permanentes, zonas residenciales, ríos y mares o lagos. Cada clase contiene entre 2,000 y 3,000 imágenes (Pasture es la menor, con 2,000, y varias tienen 3,000), por lo que cumple con la cantidad de ejemplos solicitada para el proyecto (Helber et al., 2019).

La Figura 1 muestra un ejemplo de cada clase. Algunas se reconocen por el color o el tono general, como los bosques y los mares o lagos, y otras por la textura y la forma del terreno. Las clases que más se parecen entre sí son las de vegetación y cultivos: los cultivos anuales y permanentes y la vegetación herbácea comparten tonos verdes y marrones, y se distinguen sobre todo por la distribución de las parcelas. Esto se confirma en los resultados (Figura 11): la CNN confundió 36 de las 375 imágenes de cultivos permanentes con vegetación herbácea.

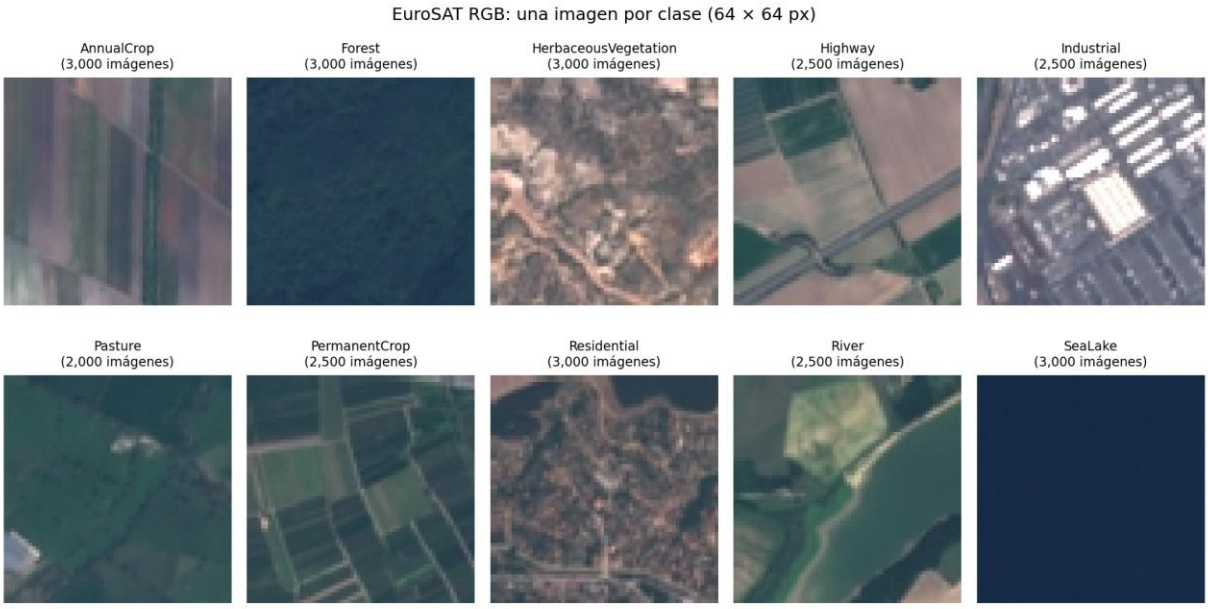


Figura 1. Una imagen por clase de EuroSAT RGB, con el número de imágenes de cada clase.

Algoritmos y aplicaciones posibles

Comparamos una red densa o MLP con una CNN propia para analizar cómo cambia la clasificación de imágenes de EuroSAT según la arquitectura utilizada. También revisamos la transferencia de aprendizaje como una alternativa para continuar el proyecto.

Red densa o MLP como línea base. Cada imagen de 64 × 64 píxeles y tres canales se convirtió en un vector de 12,288 valores. Este vector pasó por capas densas con activación ReLU y una salida de diez clases. La MLP permitió establecer una referencia inicial. Aunque conservó los valores de los píxeles, su arquitectura no incorporó explícitamente las relaciones de vecindad que sí aprovecha una CNN.

CNN propia. Entrenamos una red con bloques convolucionales, BatchNorm, ReLU y MaxPooling, seguidos de un clasificador de diez clases. Las convoluciones permitieron aprender patrones locales como bordes, texturas y distribuciones de construcciones o vegetación. Para evaluar el aumento de datos entrenamos dos versiones de la misma CNN, una sin transformaciones y otra con volteos y rotaciones aplicados únicamente al entrenamiento. Ambas mantuvieron Dropout en el clasificador.

Transferencia de aprendizaje. Consideramos ResNet-50 con pesos preentrenados en ImageNet como una alternativa, aunque no se implementó en el proyecto. Sus conexiones residuales facilitan el entrenamiento de redes profundas (He et al., 2016). Helber et al. (2019) reportaron 98.57 % de exactitud con esta arquitectura en EuroSAT RGB. Tomamos ese resultado como antecedente porque una comparación directa requeriría reproducir sus condiciones de evaluación. También consideramos MobileNetV2 como una opción de menor costo computacional (Sandler et al., 2018).

Este tipo de clasificación puede apoyar aplicaciones como las siguientes:

Actualizar mapas de uso de suelo, como los del servicio de monitoreo terrestre de Copernicus (Copernicus Land Monitoring Service, s. f.).

Estimar superficie cultivada y apoyar el seguimiento de los cultivos.

Identificar el crecimiento urbano e industrial para la planeación de ciudades.

Vigilar cambios ambientales, como pérdida de bosques o cambios en cuerpos de agua.

Comparar zonas antes y después de un desastre, como una inundación.

Cada imagen de EuroSAT es un parche al que el modelo asigna una sola categoría. Una escena satelital más grande podría dividirse en parches georreferenciados y clasificarse para construir un mapa por bloques. Este procedimiento no identifica la clase de cada píxel ni delimita con precisión parcelas o edificios, por lo que su aplicación en un contexto real requeriría validación adicional.

Metodología

Preparamos las imágenes de EuroSAT RGB y entrenamos una MLP como modelo base y una CNN propia. En ambos modelos usamos retropropagación para calcular los gradientes y Adam para actualizar los parámetros. La implementación final se hizo en PyTorch para mantener el mismo esquema de entrenamiento y evaluación en las dos arquitecturas, con ejecución en CPU y GPU.

Arquitecturas implementadas

1 Red densa o MLP

Para empezar usamos una red densa con dos capas ocultas de 256 y 128 neuronas. Recibió las imágenes RGB de 64 × 64 píxeles con valores entre 0 y 1. Después cada imagen se convirtió en un vector de 12,288 valores que pasó por las capas ocultas y una salida de diez clases.

Elegimos esta arquitectura porque permitió aplicar lo visto en clase sobre neuronas, funciones de activación y retropropagación. También sirvió como referencia para comparar el desempeño de la CNN.

Etapa	Capas	Salida
Entrada	Imagen RGB, valores divididos entre 255	64 × 64 × 3
Aplanado	Flatten	12,288
Capa oculta 1	Linear 256, ReLU, Dropout p = 0.5	256
Capa oculta 2	Linear 128, ReLU, Dropout p = 0.5	128
Salida	Linear 10, logits	10

Tabla 1. Arquitectura de la MLP de referencia.

Flatten reorganizó los valores de la imagen en un vector sin eliminar píxeles ni añadir parámetros. Cada neurona de la primera capa oculta se conectó con los 12,288 valores de entrada. Por eso la red tuvo 3,180,170 parámetros entrenables, concentrados principalmente en esa primera capa.

Usamos ReLU para introducir no linealidad. Su derivada es uno cuando la preactivación es positiva, lo que ayuda a propagar el gradiente en esa región. Los pesos de las capas ocultas se inicializaron con el método de He y los sesgos en cero (He et al., 2015). También aplicamos Dropout de 0.5 durante el entrenamiento para reducir la dependencia entre neuronas (Srivastava et al., 2014).

La última capa devolvió diez logits, es decir valores sin normalizar. No añadimos softmax antes de la pérdida porque nn.CrossEntropyLoss incorpora la operación equivalente mediante log-softmax. Para obtener la clase predicha seleccionamos el logit más alto.

2 CNN propia

Entrenamos una CNN con tres bloques convolucionales y un clasificador denso. Elegimos esta arquitectura porque las clases de EuroSAT se distinguen por sus texturas y por la distribución del terreno, como las hileras de un cultivo o las construcciones de una zona residencial. Los filtros compartidos permiten buscar un mismo patrón en distintas regiones de la imagen (LeCun et al., 1998).

En la siguiente tabla las dimensiones de los mapas se muestran como canales × alto × ancho, sin incluir el tamaño del lote.

Etapa	Capas	Salida
Entrada	Imagen RGB, valores divididos entre 255	3 × 64 × 64
Bloque 1	2 × (Conv2d 3 × 3, 32 filtros, BatchNorm, ReLU) y MaxPool 2 × 2	32 × 32 × 32
Bloque 2	2 × (Conv2d 3 × 3, 64 filtros, BatchNorm, ReLU) y MaxPool 2 × 2	64 × 16 × 16
Bloque 3	2 × (Conv2d 3 × 3, 128 filtros, BatchNorm, ReLU) y MaxPool 2 × 2	128 × 8 × 8
Aplanado	Flatten	8,192
Clasificador	Linear 128, ReLU, Dropout p = 0.5	128
Salida	Linear 10, logits	10

Tabla 2. Arquitectura de la CNN implementada.

Cada bloque utilizó dos convoluciones de 3 × 3 con relleno para conservar el tamaño espacial, seguidas de MaxPooling de 2 × 2. Dos convoluciones de 3 × 3 consecutivas cubren un campo receptivo de 5 × 5 y añaden una activación intermedia (Simonyan y Zisserman, 2015). El MaxPooling redujo a la mitad el alto y ancho de los mapas y disminuyó la sensibilidad a pequeños desplazamientos (Goodfellow et al., 2016).

Después de cada convolución aplicamos BatchNorm y luego ReLU. BatchNorm normalizó las activaciones con las estadísticas del minilote durante el entrenamiento y ayudó a estabilizar la optimización (Ioffe y Szegedy, 2015).

El desvanecimiento del gradiente ocurre cuando los gradientes se vuelven muy pequeños al retropropagarse por las capas, haciendo más lento el aprendizaje de las primeras. ReLU y la inicialización de He ayudan a mantener la escala de la señal, aunque no garantizan que este problema desaparezca.

Al final Flatten convirtió el mapa de 128 × 8 × 8 en un vector de 8,192 valores que pasó al clasificador. La red tuvo 1,337,898 parámetros entrenables. Dropout se mantuvo en ambas versiones de la CNN y la comparación de regularización evaluó el efecto de añadir aumento de datos.

3 Transferencia de aprendizaje como trabajo futuro

Consideramos ResNet-50 preentrenada en ImageNet como una posible continuación del proyecto. Sus conexiones residuales permiten sumar la entrada de un bloque a su transformación y ofrecen una ruta adicional para propagar la información y los gradientes (He et al., 2016). Esta alternativa no se implementó porque el trabajo se centró en comparar la MLP y la CNN propia.

Para el trabajo futuro fijamos una resolución de entrada de 224 × 224 píxeles, redimensionando las imágenes originales de 64 × 64. Se conservaría la base convolucional preentrenada y se reemplazaría el clasificador por una salida de diez clases. Primero se entrenaría el nuevo clasificador con la base congelada y después se probaría un ajuste fino. El preprocesamiento utilizaría la normalización correspondiente a los pesos de ImageNet en PyTorch. La comparación se haría con las mismas particiones y métricas utilizadas en los modelos implementados.

Preprocesamiento de las imágenes

Trabajamos con la copia de EuroSAT RGB de Kaggle, organizada en carpetas por clase con imágenes JPEG de 64 × 64 píxeles (ryanholbrook, s. f.). El procesamiento fue el siguiente:

- Partición estratificada. Separamos 70 % para entrenamiento, 15 % para validación y 15 % para prueba, con semilla 42. Obtuvimos 18,900, 4,050 y 4,050 imágenes respectivamente. La separación conservó las proporciones de las diez clases y se hizo antes de aplicar las transformaciones.
- Tamaño. La MLP y la CNN utilizaron las imágenes originales de 64 × 64 píxeles sin redimensionarlas. La resolución de 224 × 224 quedó reservada para el trabajo futuro con ResNet-50.
- Normalización. Convertimos las imágenes a float32 y dividimos sus valores entre 255 para llevarlos al intervalo [0, 1].
- Formato. La MLP aplanó cada imagen en 12,288 valores. La CNN conservó la información espacial en tensores con tres canales y dimensiones de 64 × 64.
- Etiquetas. Representamos las clases con índices enteros de 0 a 9. Usamos estas etiquetas directamente con nn.CrossEntropyLoss, sin convertirlas a one-hot en el código.
- Aumento de datos. En la CNN regularizada aplicamos rotaciones de 0°, 90°, 180° o 270° y reflexiones únicamente sobre entrenamiento. La MLP y la CNN de control no utilizaron aumento. Las imágenes de validación y prueba conservaron su orientación original.

Carga y lotes. Cargamos las imágenes desde las carpetas por clase y las convertimos a tensores de PyTorch. El entrenamiento procesó lotes de 64 imágenes y mezcló el orden de los ejemplos en cada época.

Función de pérdida

Usamos entropía cruzada multiclase porque cada imagen tuvo una sola etiqueta entre diez categorías mutuamente excluyentes. Para un lote de N imágenes la pérdida se expresó como:

$$LCE = -1N\sum n = 1N\sum j = 110tn, j \log(pn, j)$$

Aquí tn, j vale 1 cuando la imagen n pertenece a la clase j y 0 en las demás clases. Esta representación se utiliza para expresar la función matemáticamente, mientras que en el código usamos índices enteros. El término pn, j representa la probabilidad estimada mediante softmax para esa clase (Goodfellow et al., 2016).

Como cada imagen pertenece a una sola categoría, la pérdida individual se simplifica al logaritmo negativo de la probabilidad asignada a la clase correcta:

$$\ell n = -\log(pn, yn)$$

En el código pasamos los logits directamente a nn.CrossEntropyLoss. Esta función combinó log-softmax con la pérdida de la clase correcta, evitando aplicar softmax por separado antes de calcular el error.

Para una imagen, la derivada de su pérdida respecto al logit de la clase j fue:

$$\partial \ell n \partial zn, j = pn, j - tn, j$$

Esta expresión representa la diferencia entre la probabilidad estimada y la etiqueta real. Para la pérdida promedio del lote el gradiente incluye además el factor 1N. Si la red asigna poca probabilidad a la clase correcta, la pérdida aumenta y la retropropagación calcula cómo ajustar los pesos para reducir ese error.

EuroSAT tiene entre 2,000 y 3,000 imágenes por clase, por eso además de la exactitud revisamos el F1 por clase: Pasture, la clase con menos ejemplos, obtuvo 0.929, mientras PermanentCrop y HerbaceousVegetation alcanzaron 0.867 y 0.886 respectivamente (Tabla 7). La pérdida se utilizó sin ponderar y estas diferencias de F1 no demuestran que el desbalance sea la causa de los errores, por lo que una versión ponderada quedó como prueba futura a seleccionar mediante validación.

Entrenamiento, Experimentación Inicial y Regularización

Entrenamos la red densa (Tabla 1) utilizando el optimizador Adam y la función de pérdida de entropía cruzada multiclase, trabajando con una partición estratificada de EuroSAT RGB con semilla fija (42) y lotes de 64 imágenes. Todo el preprocesamiento consistió en convertir los pixeles a punto flotante y dividirlos entre 255, sin aplicar aumento de datos inicial.

Para evaluar este modelo base (R1), probamos cuatro configuraciones alterando una variable a la vez: la tasa de aprendizaje (R2), el ancho de las capas ocultas (R3) y el número de épocas (R4). Los

hiperparámetros probados se detallan en la Tabla 3, y los resultados finales de validación en la Tabla 4. Cabe destacar que los resultados de R1 a R4 se reportan con la corrida original de CPU, y que su repetición en GPU arrojó valores distintos (por ejemplo, R3 terminó en 25.48 % de validación y no en 17.46 %), por lo que las diferencias numéricas exactas entre estas configuraciones iniciales son puramente indicativas.

ID	Cambio	Capas ocultas	Tasa de aprendizaje	Épocas	Parámetros
R1	Base	256-128	0.001	20	3,180,170
R2	Tasa de aprendizaje menor	256-128	0.0001	20	3,180,170
R3	Más neuronas	512-256	0.001	20	6,425,866
R4	Más épocas	256-128	0.001	50	3,180,170

Tabla 3. Configuraciones probadas

ID	Exactitud ent.	Pérdida ent.	Exactitud val.	Pérdida val.	Mejor exactitud val. (época)
R1	21.62 %	2.0355	26.96 %	1.9339	28.05 % (15)
R2	34.81 %	1.7048	42.02 %	1.6135	44.25 % (18)
R3	23.61 %	2.0070	17.46 %	2.0542	26.81 % (10)
R4	21.96 %	2.0285	20.64 %	1.9668	28.05 % (15)

Tabla 4. Resultados en la última época y mejor exactitud de validación

Al analizar las curvas de aprendizaje preliminares (Figuras 2 a 5), la pérdida del modelo base R1 bajó de 2.20 a 2.04, separándose por poco del azar($\ln 10 \approx 2.30$), lo que evidencia subajuste y un entrenamiento inestable debido a la tasa de aprendizaje, no un sobreajuste. Reducir la tasa de aprendizaje a 0.0001 (R2) suavizó las curvas y mejoró el desempeño, alcanzando un 44.25 % como exactitud máxima. Entrenar una red más grande

(R3) o con más épocas (R4) no logró corregir la inestabilidad propia de la tasa de aprendizaje alta.

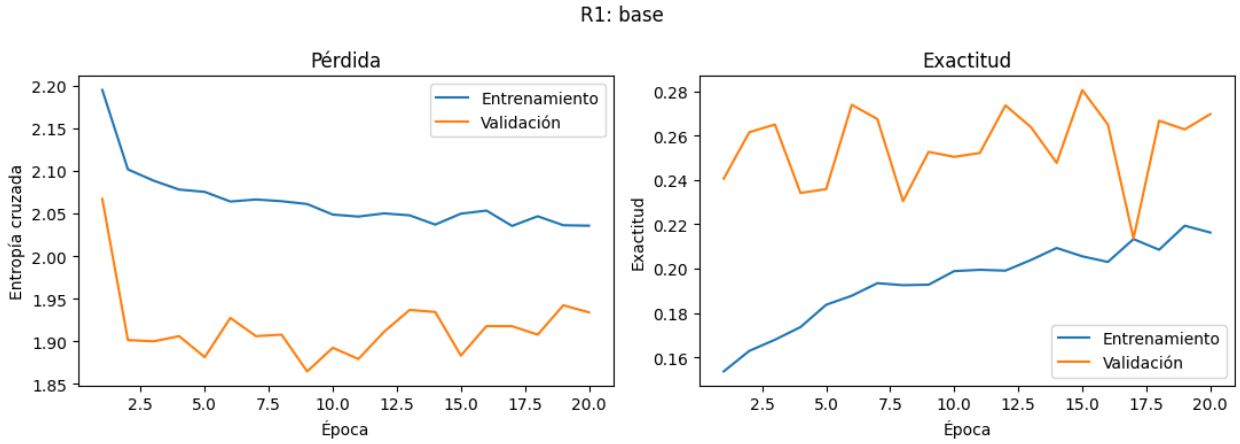


Figura 2. Curvas de aprendizaje de R1 (256-128, lr = 0.001, 20 épocas).

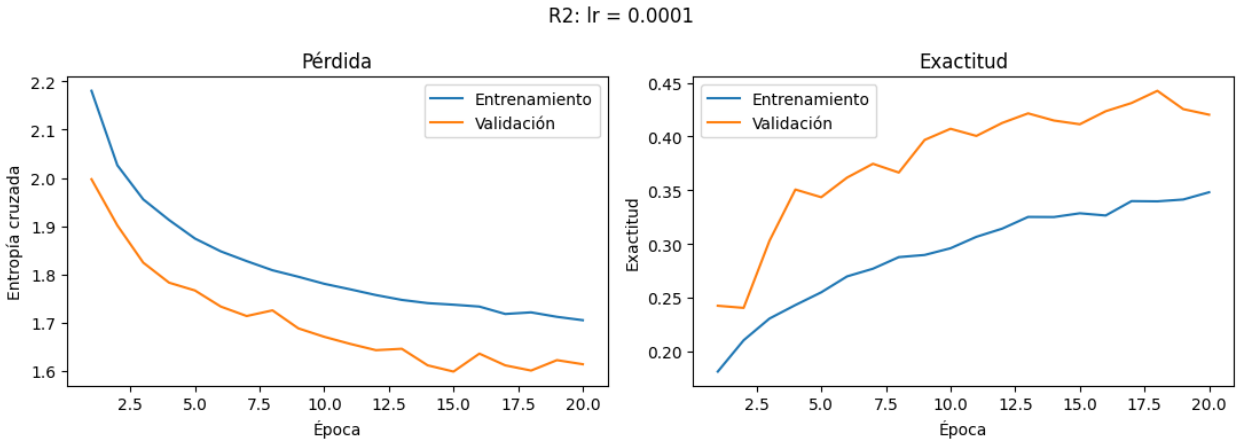


Figura 3. Curvas de aprendizaje de R2 (lr = 0.0001).

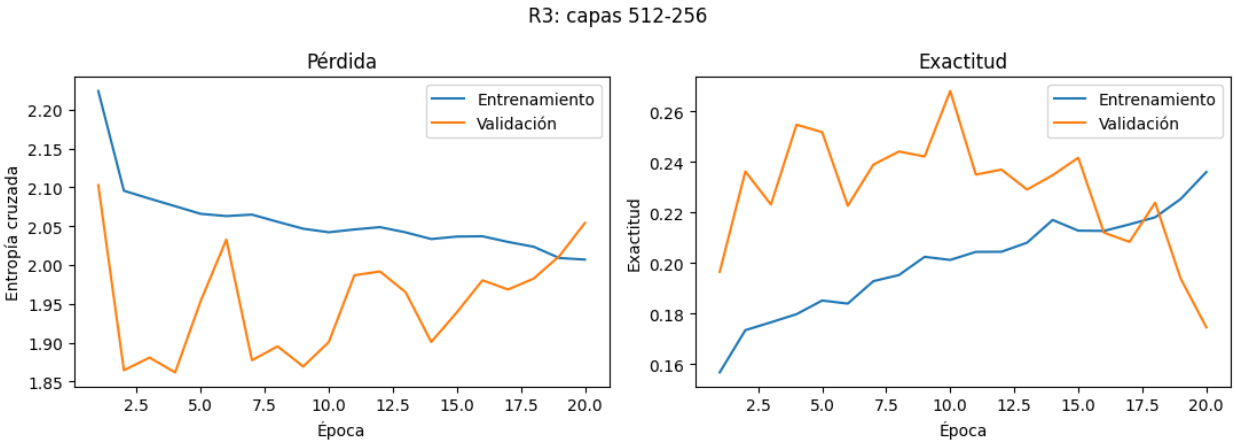


Figura 4. Curvas de aprendizaje de R3 (capas 512-256).

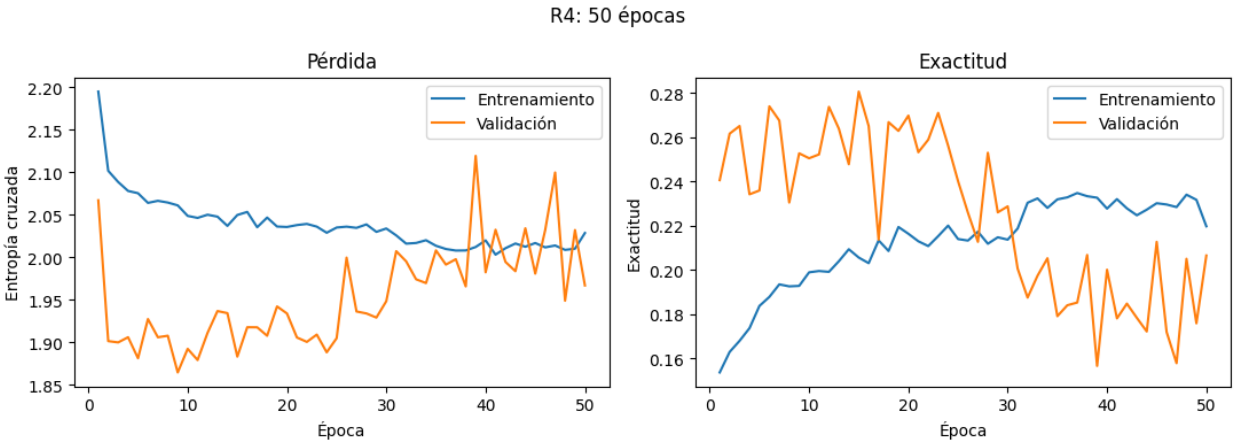


Figura 5. Curvas de aprendizaje de R4 (50 épocas).

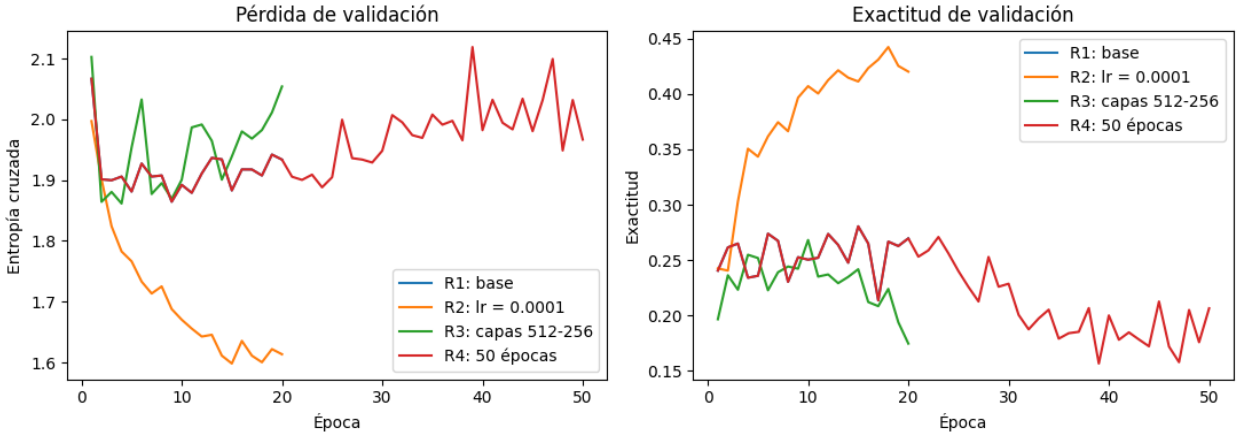


Figura 6. Pérdida y exactitud de validación de las cuatro corridas.

Para identificar posibles señales de sobreajuste, evaluamos las curvas de la mejor configuración densa (R2). Dado que la exactitud de validación se mantenía por encima del entrenamiento debido a que Dropout se desactiva al validar, decidimos medir también el entrenamiento en modo evaluación (Figura 7). Esto confirmó que las curvas se mantienen cercanas, apuntando principalmente a un aprendizaje insuficiente.

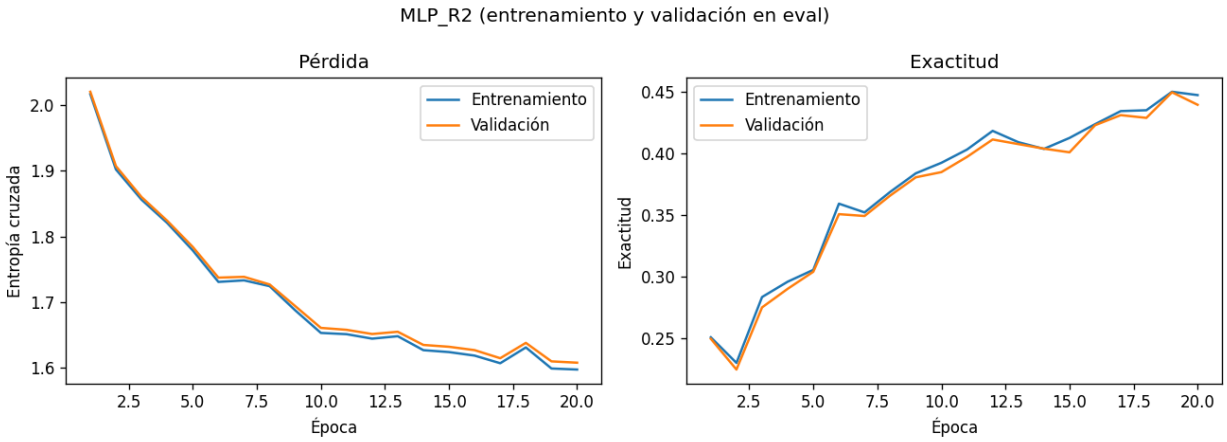


Figura 7: Curvas de aprendizaje de la MLP (R2), con entrenamiento y validación medidos en modo de evaluación

Para el entrenamiento de la red convolucional , se mantuvo Dropout (p=0.5) y se aplicó aumento de datos (rotaciones de 90° y reflexiones) exclusivamente al conjunto de entrenamiento para ampliar las variaciones visuales. La CNN de control (sin aumento) alcanzó 97.40 % de exactitud en entrenamiento frente a 91.28 % en validación en su época 18 (Figura 8), una separación creciente compatible con sobreajuste. En contraste, la CNN con aumento de datos (Figura 9) acercó ambas curvas y mejoró el desempeño sobre datos no vistos, logrando una exactitud de validación de 92.74 % y reduciendo la brecha a solo 1.42 puntos porcentuales en la época 20 (Tabla 5).

Modelo	Época seleccionada	Exactitud de entrenamiento	Exactitud de validación	Pérdida de validación
CNN de control	18	97.4 %	91.28 %	0.2518
CNN con aumento de datos	20	94.16 %	92.74 %	0.2156

Tabla 5. Resultados de los momentos seleccionados para cada red convolucional.

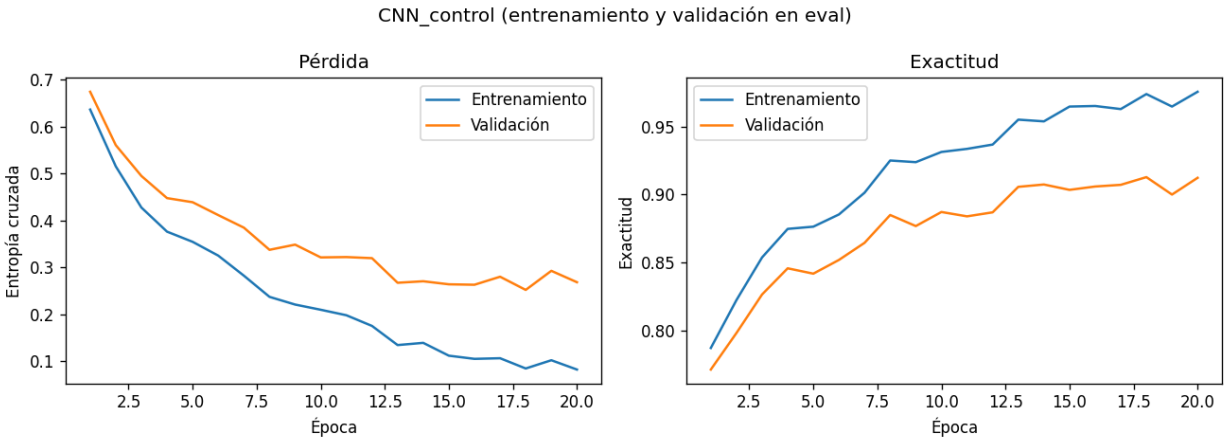


Figura 8: Curvas de aprendizaje de la CNN sin aumento de datos

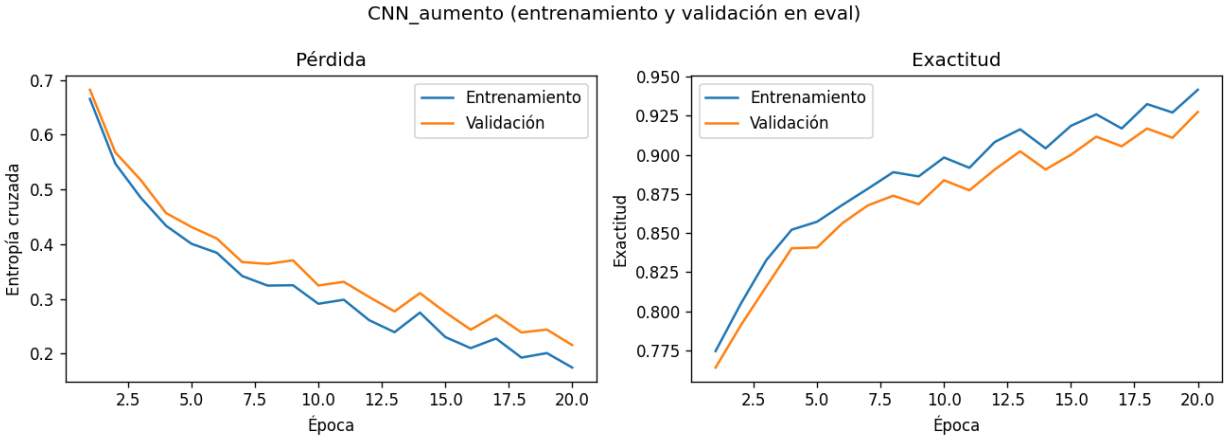


Figura 9: Curvas de aprendizaje de la CNN con aumento de datos

Resultados y evaluación

Comparamos la MLP de referencia (R2) contra las dos variantes de CNN bajo el mismo protocolo: partición 70/15/15 (semilla 42), optimizador Adam (lr=0.0001), lotes de 64 y 20 épocas. Como se observa en la Figura 10, la arquitectura CNN superó a la MLP desde la primera época (77.11 % de exactitud contra 24.99 %) utilizando un 58 % menos de parámetros (1.33 vs 3.18 millones).

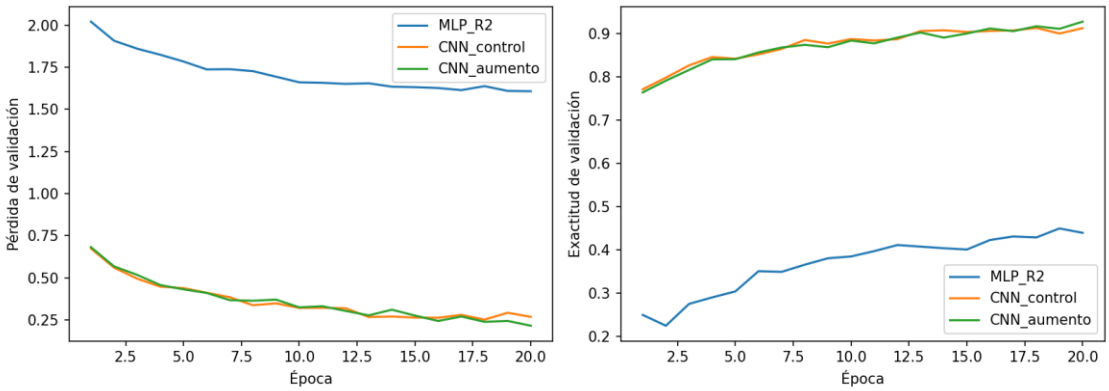


Figura 9: Pérdida y exactitud de validación de la MLP (R2) y de las dos CNN.

Seleccionamos la CNN con aumento de datos por su menor pérdida y mayor exactitud en validación, y realizamos la consulta final sobre el conjunto de prueba (4,050 imágenes) apartado desde el inicio.

Modelo	**Parámetros**	**Exactitud**	**Pérdida**	**F1 macro**	**F1 ponderado**
MLP(R2)	3,180,170	45.19%	1.5999	0.418	0.425
CNN con aumento	1,337,898	93.01%	0.2055	0.928	0.930

Tabla 6. Métricas finales en el conjunto de prueba (4,050 imágenes).

La CNN superó a la MLP en casi 48 puntos porcentuales de exactitud en prueba, incrementando el F1 macro de 0.418 a 0.928 (Tabla 6). Este salto drástico responde a la forma en que ambas procesan la información: la MLP aplana la imagen en un vector de 12,288 valores, perdiendo el contexto espacial, por lo que solo logra clasificar correctamente texturas dominantes por color, como bosques (F1 de 0.707) (Figura 11). En contraste, los filtros compartidos y la agrupación de la CNN detectan jerárquicamente estructuras locales (trazos de calles o hileras de cultivos), lo que minimiza el error sistemático y domina ampliamente a través de todas las categorías (Figura 12).

Clase	**Imágenes**	**MLP F1**	**CNN precisión**	**CNN recall**	**CNN F1**
AnnualCrop	450	0.446	0.903	0.933	0.918

Forest	450	0.707	0.970	0.996	0.983
HerbaceousVegetation	450	0.246	0.849	0.927	0.886
Highway	375	0.076	0.958	0.845	0.898
Industrial	375	0.624	0.952	0.957	0.955
Pasture	300	0.409	0.905	0.953	0.929
PermanentCrop	375	0.255	0.886	0.848	0.867
Residential	450	0.443	0.980	0.960	0.970
River	375	0.361	0.909	0.883	0.896
SeaLake	450	0.615	0.989	0.976	0.982

Tabla 7. Métricas por clase en prueba.

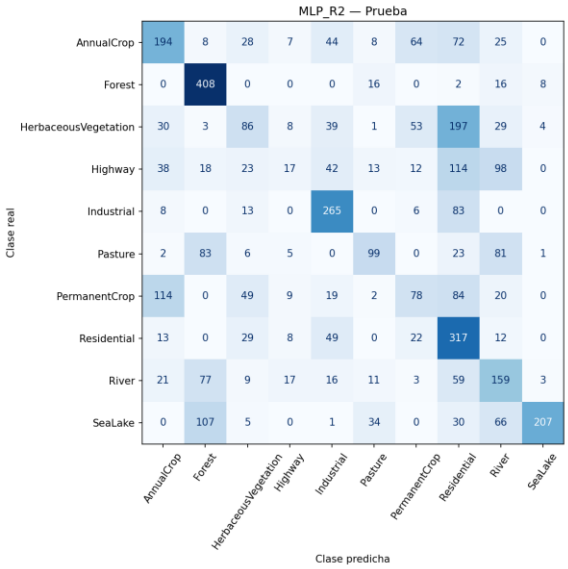


Figura 11: Matriz de confusión de la MLP (R2) en el conjunto de prueba.

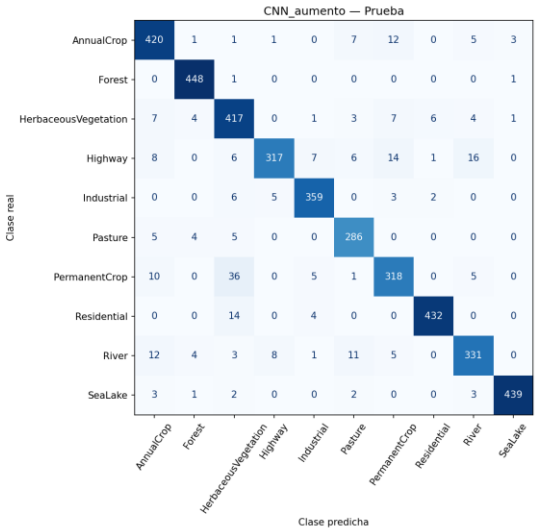


Figura 12: Matriz de confusión de la CNN con aumento de datos en el conjunto de prueba.

Modelo Elegido y Limitaciones

El modelo definitivo es la CNN entrenada con aumento de datos, registrando un 93.01 % de exactitud y 0.2055 de pérdida de prueba, respaldando la capacidad de las convoluciones para diferenciar categorías que dependen de su topología y distribución estructural. No obstante, estos resultados operan bajo ciertas limitaciones metodológicas: el diseño utilizó una única partición y semilla, lo que impide asegurar la estabilidad estadística de mejoras marginales (como el 1.46% ganado por el aumento de datos). Adicionalmente, la mejor exactitud de la CNN se dio en la última época configurada (la número 20), sugiriendo que la red no terminó de converger y podría optimizarse extendiendo el entrenamiento. Finalmente, dado que la comparativa arquitectónica no fue completamente aislada (la CNN incluye Normalización por Lotes y la MLP no), una porción del salto de rendimiento podría derivar de estas integraciones más allá de la convolución espacial.

Limitaciones

Los resultados de este proyecto tienen límites que conviene tener presentes al interpretarlos.

Diseño del experimento. Usamos una sola semilla (42) y una sola partición 70/15/15. No sabemos cuánto cambiarían las cifras con otra semilla u otra partición. La diferencia de 1.46 puntos porcentuales entre la CNN con y sin aumento de datos es pequeña y podría moverse. La diferencia de casi 48 puntos entre la MLP y la CNN es tan grande que es poco probable que dependa de la semilla, pero no lo comprobamos.

Entrenamiento. La mejor época de la CNN con aumento fue la última (la 20), por lo que 20 épocas pudieron quedarse cortas. No probamos más épocas con parada temprana. La MLP tampoco terminó de converger: R2 seguía mejorando en la época 20. Su resultado (45.19 % en prueba) es entonces un piso, no el máximo que esa arquitectura puede alcanzar. Además, las corridas con tasa de aprendizaje 0.001 (R1, R3 y R4) fueron inestables. Al repetirlas en GPU sus valores cambiaron (por ejemplo, R3 terminó en 25.48 % y no en 17.46 %), así que esas diferencias son sólo indicativas.

Comparación entre arquitecturas. La CNN usa BatchNorm y la MLP no, y no entrenamos una MLP con BatchNorm. Por eso no podemos atribuir toda la diferencia a las convoluciones. Tampoco entrenamos una CNN sin Dropout, así que no aislamos cuánto aporta Dropout frente al aumento de datos. Este último solo se comparó contra un control con Dropout.

Modelos no implementados. ResNet-50 con preentrenamiento quedó como trabajo futuro. El 98.57 % de Helber et al. (2019) se obtuvo con otro protocolo de evaluación, así que no es comparable de forma directa con nuestro 93.01 %.

Datos y uso real. El aumento de datos (rotaciones y reflexiones) no crea observaciones independientes. El conjunto contiene solo imágenes de Europa, de 64 × 64 píxeles en RGB. La clasificación es por parche y no por píxel, por lo que usarla en un mapa real requeriría validación adicional. Por clase, las más débiles de la CNN fueron PermanentCrop (F1 0.867), River (0.896) y Highway (0.898).

Pruebas pendientes. Con más tiempo haríamos: (1) varias semillas y particiones, con promedio y desviación; (2) una CNN sin Dropout; (3) una MLP con BatchNorm; (4) más épocas con parada temprana; (5) ResNet-50 con el mismo protocolo de partición.

Conclusiones

La pregunta del proyecto era si una CNN clasifica mejor que una MLP las imágenes de EuroSAT. La respuesta, con el mismo protocolo, es que sí, y por mucho. En el conjunto de prueba (4,050 imágenes) la CNN con aumento de datos obtuvo 93.01 % de exactitud y 0.928 de F1 macro, contra 45.19 % y 0.418 de la MLP. La diferencia es de 47.83 puntos porcentuales, y la pérdida bajó de 1.5999 a 0.2055. La CNN superó a la MLP desde la primera época (77.11 % contra 24.99 % en validación).

La ventaja no viene de tener más parámetros: la CNN tiene 1,337,898, un 58 % menos que los 3,180,170 de la MLP. Lo más probable es que venga de cómo procesa la imagen. Sus filtros reutilizan los mismos pesos en toda la imagen y aprovechan la vecindad de los píxeles. La MLP, en cambio, recibe los

12,288 valores como un solo vector. Los errores de la MLP apoyan esta idea: acierta en clases con un color característico, como Forest, y falla en las que dependen de la textura, como Highway (F1 de 0.076).

En la MLP, la variable más influyente fue la tasa de aprendizaje. Bajarla de 0.001 a 0.0001 subió la exactitud de validación de 26.96 % a 42.02 % sin cambiar la arquitectura. Aumentar el tamaño de la red o las épocas no ayudó con la tasa alta.

En la CNN, el aumento de datos redujo el sobreajuste. La brecha entre entrenamiento y validación bajó de 6.12 a 1.42 puntos porcentuales, la exactitud de validación subió de 91.28 % a 92.74 % y la pérdida bajó de 0.2518 a 0.2156.

Estas conclusiones valen para nuestra partición y nuestra semilla, y no podemos atribuir toda la diferencia a las convoluciones, porque solo la CNN usa BatchNorm. Aun así, la brecha es tan grande que respalda la elección de una arquitectura convolucional para este problema. Como siguiente paso, lo más útil sería repetir los experimentos con varias semillas y comparar con ResNet-50 en las mismas condiciones.

Reflexión (equipo)

Este proyecto nos enseñó que un buen resultado depende tanto de cómo se hace el experimento como del modelo que se elige.

Lo primero que aprendimos fue a leer las curvas de aprendizaje antes de cambiar algo. La MLP parecía sobreajustar, pero las curvas mostraron que en realidad aprendía poco: era un problema de optimización. Bajar la tasa de aprendizaje de 0.001 a 0.0001 mejoró la exactitud de validación de 26.96 % a 42.02 % sin tocar la arquitectura, y agrandar la red no ayudó. Sin las curvas habríamos corregido el problema equivocado.

También entendimos que una métrica depende de las condiciones en que se calcula. La exactitud de validación salía más alta que la de entrenamiento porque Dropout solo está activo al entrenar. Por eso medimos el entrenamiento también en modo evaluación, para comparar ambas métricas con los mismos pesos.

La comparación entre la MLP y la CNN nos mostró el valor de usar el mismo protocolo: mismas particiones, mismo optimizador, misma tasa de aprendizaje y mismas épocas. Así, la diferencia de 47.83 puntos porcentuales en prueba (93.01 % contra 45.19 %) se puede atribuir a la arquitectura y no a

condiciones distintas. Vimos además que la CNN, con 58 % menos parámetros, rindió mucho más, así que más parámetros no significa mejor modelo.

Aprendimos a proteger el conjunto de prueba. Elegimos el modelo con validación y consultamos la prueba una sola vez, sin ajustar nada después. Eso hace más confiables las cifras finales.

Como equipo, también aprendimos a ser honestos con los límites del trabajo. Usamos una sola semilla y una sola partición, la MLP no terminó de converger, no probamos una MLP con BatchNorm y no implementamos ResNet-50. Nos pasó además que no guardamos los pesos de R2 y tuvimos que reentrenarla en GPU, lo que cambió algunos valores. Esto nos enseñó a guardar los pesos de cada corrida desde el inicio y a reportar qué se hizo y qué no.

Si repitiéramos el proyecto, correríamos varias semillas desde el principio y compararíamos una MLP con BatchNorm, para separar cuánto aportan las convoluciones y cuánto la normalización. También probaríamos ResNet-50 con el mismo protocolo, para que la comparación con la literatura fuera directa.

Reflexiones (individuales)

Lenin: Este reto me sirvió para aplicar lo que vimos de redes neuronales en un problema completo: clasificar imágenes satelitales de EuroSAT en diez tipos de terreno. Construir el modelo resultó ser la parte más corta. Lo demás fue entrenarlo con cuidado, evaluarlo sin maquillar nada y explicar lo que salía.

Empecé con una red densa como modelo base, para tener una referencia antes de pasar a algo más complejo. Los primeros resultados fueron malos: con la tasa de aprendizaje inicial, la exactitud de validación apenas pasó del 28 %. En vez de cambiar la arquitectura al azar, movimos una variable a la vez (tasa de aprendizaje, ancho de las capas, número de épocas) y lo que más ayudó fue bajar la tasa de 0.001 a 0.0001. Ahí vi que un mal resultado puede venir de cómo se entrena y no de cómo se diseñó la red, y que cambiar una sola cosa por prueba es lo que permite saber qué causó qué.

También aprendí a desconfiar de los resultados que se ven bien. En la red densa, la exactitud de validación quedó por encima de la de entrenamiento, y a primera vista parecía buena señal. Al revisarlo, resultó que las dos se medían en condiciones distintas: con Dropout activo en una y apagado en la otra. No demostraba que el modelo generalizara mejor. Con la CNN comparamos la misma red con y sin

aumento de datos, y el aumento redujo la brecha entre entrenamiento y validación de 6.12 a 1.42 puntos porcentuales.

Al final, la CNN llegó a 93.01 % de exactitud en prueba, contra 45.19 % de la red densa, y con menos de la mitad de los parámetros. Confío en ese número porque la comparación fue controlada y el conjunto de prueba se usó una sola vez. Aun así, tiene límites. Usamos una sola semilla y una sola partición. La mejor época de la CNN fue la última, así que seguramente faltó entrenar más. Y la CNN lleva BatchNorm y la red densa no, de modo que no toda la mejora se puede atribuir a las convoluciones.

Un modelo así serviría para actualizar mapas de uso de suelo, estimar la superficie cultivada, detectar el crecimiento urbano e industrial o vigilar cambios ambientales como la pérdida de bosques o los cambios en cuerpos de agua. Pero clasifica parches completos, no píxeles, así que usarlo en la práctica exigiría más validación.

Me quedo con que entender el problema, probar con orden y reportar los límites pesa tanto como la exactitud final.

Link al repositorio de GitHub:

https://github.com/LeninCabCol/Reto-Redes-Neuronales-y-Aprendizaje-Profundo

Referencias/Bibliografía

- Agencia Espacial Europea. (s. f.). Sentinel-2. https://www.esa.int/Applications/Observing_the_Earth/Copernicus/Sentinel-2
- Copernicus Land Monitoring Service. (s. f.). *Copernicus Land Monitoring Service*. https://land.copernicus.eu/en
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. MIT Press.
- He, K., Zhang, X., Ren, S., & Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification. En Proceedings of the IEEE International Conference on Computer Vision (ICCV) (pp. 1026-1034).

https://openaccess.thecvf.com/content_iccv_2015/html/He_Delving_Deep_into_ICCV_2015_paper.html

- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. En *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778). IEEE. https://doi.org/10.1109/CVPR.2016.90
- Helber, P., Bischke, B., Dengel, A., & Borth, D. (2019). EuroSAT: A novel dataset and deep learning benchmark for land use and land cover classification. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing, 12*(7), 2217–2226. https://doi.org/10.1109/JSTARS.2019.2918242
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. En F. Bach & D. Blei (Eds.), Proceedings of the 32nd International Conference on Machine Learning (Vol. 37, pp. 448-456). PMLR. https://proceedings.mlr.press/v37/ioffe15.html
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. En International Conference on Learning Representations (ICLR 2015). https://arxiv.org/abs/1412.6980
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE, 86(11), 2278-2324. https://doi.org/10.1109/5.726791
- Li, D., Lee, T., Vázquezet, M., Tsoi, N. (2025). Aligning Multiclass Neural Network Classifier Criterion with Task Performance Metrics. *Arxiv*. 2405.20954
- ryanholbrook. (s. f.). *EuroSAT* [Conjunto de datos]. Kaggle. https://www.kaggle.com/datasets/ryanholbrook/eurosat
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. En *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520). IEEE. https://doi.org/10.1109/CVPR.2018.00474
- Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. En International Conference on Learning Representations (ICLR 2015). https://arxiv.org/abs/1409.1556
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(56), 1929-1958. https://jmlr.org/papers/v15/srivastava14a.html

- TensorFlow. (2026, 22 de septiembre). *Eurosat*. TensorFlow Datasets. https://www.tensorflow.org/datasets/catalog/eurosat
- Zhou et al. (2019). MPCE: A Maximum Probability Based Cross Entropy Loss Function for Neural Network Classification. *ieeexplore*. IEEE Xplore Full-Text PDF: